

FastAPI Course — Complete Study Notes

Chapters 1–7 · OTLMS Python/FastAPI Course

Prepared for Swapnil — for reading, writing, and hands-on practice

What's inside this document:

- Every concept from Chapters 1–7, explained in plain, simple language
- Code examples with line-by-line breakdowns
- Tables for quick lookup (status codes, validation constraints, etc.)
- A 'Practice' section at the end of every chapter — write-in questions plus a coding exercise
- A full cheat-sheet at the very end for fast revision before a test

How to use this: Read a chapter's explanation first, then physically build the code in your own editor before checking the write-space answers. Don't just read — type it out and run it. That's how the concepts actually stick.

Contents

Chapter 1 — Your First FastAPI Application

Chapter 2 — Routes, Path Parameters & Query Parameters

Chapter 3 — Request Bodies & Pydantic Models

Chapter 4 — Deeper Validation & Splitting the Project into Layers

Chapter 5 — Response Models: Never Leak Sensitive Data

Chapter 6 — HTTP Status Codes & Custom Responses

Chapter 7 — Proper Exception Handling

Full Module — Quick Reference Cheat Sheet

Chapter 1 — Your First FastAPI Application

1. What is an API, in plain words?

Think of an API like a waiter in a restaurant. You (the client / app) don't walk into the kitchen (the server) yourself. You tell the waiter what you want, the waiter goes to the kitchen, gets it, and brings it back to you in a defined format.

In web terms: your frontend sends a request like `GET /books/42`, and the backend replies with structured data (usually JSON), like a book's title and author. FastAPI is the tool we use to build that "waiter + kitchen" system on the Python side.

2. What is FastAPI?

FastAPI is a Python web framework built specifically for creating APIs. Its biggest selling points:

- Uses normal Python type hints (`str`, `int`, `bool`) to do validation automatically
- Generates interactive API documentation for you, for free
- Runs asynchronously, so it can handle many requests efficiently
- Has built-in support for request/response validation via Pydantic

The request-response cycle, simplified:

```
Client --HTTP Request--> FastAPI App --runs your function-->
--> returns data --> FastAPI converts to JSON --HTTP Response--> Client
```

3. Why uv instead of pip + venv?

uv is a modern all-in-one tool that replaces the old combo of pip, venv, and requirements.txt. One tool handles: creating the project, managing a virtual environment, installing packages, and locking exact versions so the project behaves the same on every machine.

uv command	What it does
<code>uv init <name></code>	Creates a new project folder with starter files
<code>uv add <package></code>	Installs a dependency and updates <code>pyproject.toml</code>
<code>uv remove <package></code>	Uninstalls a dependency
<code>uv run <command></code>	Runs a command inside the project's virtual environment
<code>uv sync</code>	Makes your environment match the lockfile exactly
<code>uv lock</code>	Re-resolves and updates the lockfile

Why uv.lock matters: It records the EXACT version of every dependency. Without it, you might install FastAPI 0.100 today and your teammate installs 0.115 next month — same code, different behavior. The lockfile keeps everyone on identical versions.

4. Key files in a FastAPI + uv project

File	Purpose
pyproject.toml	Project metadata + declared dependencies
uv.lock	Exact resolved dependency versions (commit this to git)
.python-version	Which Python version this project uses
app/main.py	Where your FastAPI application code lives

5. Path Operation — the core building block

A "path operation" is simply: HTTP Method + URL Path, connected to a Python function.

```
GET /      -> a function that runs when someone visits '/'
GET /health -> a different function for '/health'
POST /books -> a function that runs when someone POSTs to '/books'
```

Common HTTP methods and what they usually mean:

Method	Typical purpose
GET	Retrieve data — should never change anything on the server
POST	Create new data
PUT	Replace an existing resource completely
PATCH	Partially update a resource
DELETE	Remove a resource

6. Your first app — full code, line by line

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Welcome to the FastAPI Learning Project"}
```

Breaking it down

- `from fastapi import FastAPI` — pulls in the FastAPI class from the library.

- `app = FastAPI()` — creates ONE central application object. Every route you add gets registered onto this object.
- `@app.get("/")` — a decorator. It tells FastAPI: "when a GET request hits the path '/', run the function right below me."
- `async def root():` — the function that actually handles the request. 'async' just means it CAN run asynchronously; you don't need to fully understand async yet.
- `return {...}` — FastAPI automatically converts this Python dictionary into a JSON response for you. You never call `json.dumps()` yourself.

7. Running it and seeing it work

```
uv run fastapi dev app/main.py
```

Then open these in your browser:

URL	What you see
<code>http://127.0.0.1:8000/</code>	Your JSON response
<code>http://127.0.0.1:8000/docs</code>	Swagger UI — interactive docs, click 'Try it out' to test endpoints
<code>http://127.0.0.1:8000/redoc</code>	ReDoc — a cleaner, read-only view of the same docs

Why /docs matters: FastAPI reads your code (the routes, the type hints) and automatically builds a full API contract from it — called an OpenAPI schema. /docs and /redoc are just two different visual renderings of that schema. You get documentation for free, and it never goes out of date because it's generated from your actual code.

8. Adding more routes

```
@app.get("/health")
async def health():
    return {"status": "healthy"}

@app.get("/about")
async def about():
    return {
        "name": "FastAPI Learning Project",
        "version": "1.0.0"
    }
```

FastAPI picks the right function purely based on (HTTP method + path) matching. There's no manual if/else routing to write.

9. Why not put everything in one main.py forever?

For 3 routes, one file is totally fine. But imagine the app grows to 50 endpoints, database models, authentication, background jobs... One giant file becomes very hard to navigate and

review. That's why, starting from Chapter 1 itself, we keep app code inside an `app/` folder — so the project has room to grow into routers, schemas, services, etc. later without a painful rewrite.

10. Common beginner mistakes

- Mixing up methods — `GET /books` and `POST /books` are two completely different routes, even though the path text is identical.
- Typo in the path — `@app.get("/health")` only matches `'/health'`, not `'/healthy'`.
- Running `uv run` from the wrong folder — always run it from the project root, the one containing `pyproject.toml`.
- Treating `'fastapi dev'` as production-ready — the dev server is for development convenience (auto-reload etc.), not for deploying to real users.

Chapter 1 — Quick Glossary

Term	Meaning
FastAPI()	Creates the central application object
Path operation	HTTP method + URL path, tied to a function
Decorator (@app.get)	Registers a function as the handler for a route
JSON response	FastAPI auto-converts Python dict/list return values to JSON
/docs	Interactive Swagger UI documentation
/redoc	Alternative read-only documentation view

PRACTICE - Answer in your own words (no peeking at the code above)

1. What is a path operation?

2. What does `@app.get("/")` actually do?

3. Why do we use `uv` instead of plain `pip`?

4. What's the difference between `pyproject.toml` and `uv.lock`?

5. Why shouldn't a growing app keep everything in one `main.py` file?

PRACTICE - Coding exercise — Personal Profile API

Build a small API (don't copy the examples above — design it yourself) with these endpoints:

- `GET /` -> a welcome message
- `GET /profile` -> name, role, experience, location
- `GET /skills` -> a list of skills
- `GET /contact` -> email + website (use fake data)

Challenge: add GET /status returning {"status": "online"}, plus one endpoint of your own design (hobbies, projects, education — your choice).

Sketch your route list and response shapes here before coding:

Chapter 2 — Routes, Path Parameters & Query Parameters

1. The problem this chapter solves

Chapter 1's routes were all fixed: GET /health always does the same thing. But real APIs need to receive information FROM the client. Example: GET /books/25 — how does our function know the client wants book number 25 specifically? That's what path and query parameters solve.

2. Path Parameters — variables inside the URL

A path parameter is a placeholder written with curly braces directly in the route path:

```
/books/{book_id}
```

Think of {book_id} as a blank slot. /books/10 fills that slot with 10, /books/250 fills it with 250 — ALL using the exact same route definition. One route can represent thousands of resources.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/books/{book_id}")
async def get_book(book_id: int):
    return {"book_id": book_id}
```

- {book_id} in the decorator tells FastAPI this part of the URL is dynamic.
- book_id: int in the function signature does TWO jobs: it links that variable to the URL slot, AND it tells FastAPI to convert the text from the URL into a real Python integer.
- Visit /books/hello (a string, not a number) and FastAPI auto-rejects it with a validation error — you never wrote an if-check for that.

Rule to remember: A path parameter is always required — it's literally part of the URL structure. You can't have /books/ mean "no book_id".

3. Query Parameters — the ?key=value part

Everything after the ? in a URL is the query string. Multiple values are joined with &:

```
/books?category=programming&limit=10
```

Path says WHICH resource. Query says HOW to filter/retrieve it.

```
@app.get("/books")
async def search_books(q: str | None = None):
    return {"search": q}
```

- `q: str | None = None` means: `q` can be a string OR missing entirely. The `'= None'` default is what makes it OPTIONAL.
- Remove the default (just `q: str`) and FastAPI makes it REQUIRED — `/search` without `q=...` will fail.

4. Defaults for pagination

```
@app.get("/books")
async def list_books(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

`/books` -> `skip=0, limit=10` (defaults). `/books?skip=20&limit=5` -> `skip=20, limit=5`. This `skip/limit` pattern is the standard way APIs implement pagination.

5. Combining path + query in one endpoint

```
@app.get("/books/{book_id}")
async def get_book(book_id: int, include_reviews: bool = False):
    return {"book_id": book_id, "include_reviews": include_reviews}
```

FastAPI figures out automatically: `book_id` matches a `{slot}` in the path, so it's a path parameter. `include_reviews` doesn't appear in the path, so it becomes a query parameter.

Bonus: bool query params accept `true/false/1/0` and FastAPI converts them for you — no manual string parsing needed.

6. Route ordering — a sneaky trap

These two routes look harmless together:

```
@app.get("/users/{user_id}")
async def get_user(user_id: int): ...

@app.get("/users/me")
async def get_current_user(): ...
```

But FastAPI checks routes top to bottom. In the order above, a request to `/users/me` would actually get caught by `/users/{user_id}` first (with `'me'` treated as `user_id`), causing a validation error — never reaching `/users/me` at all. **FIX:** always declare fixed/specific routes (like `/users/me`) BEFORE dynamic ones (`/users/{user_id}`).

7. HTTP Status Codes — quick reference

Code	Meaning	When
200	OK	Successful GET/PUT/PATCH
201	Created	Successful POST that created something

204	No Content	Success, nothing to return
400	Bad Request	Client sent malformed data
404	Not Found	Resource doesn't exist
422	Validation Error	Data failed type/field validation
500	Server Error	Something broke on the backend

8. Path vs Query — the design rule to memorize

Question	Use
"WHICH specific resource?"	Path parameter -> /books/25
"HOW should I filter/sort/paginate it?"	Query parameter -> /books?category=x&limit=10

9. Full mini-project pattern: Book Catalog API

Putting it all together — list, filter, paginate, and fetch a single item, using in-memory data (no database yet):

```
from fastapi import FastAPI

app = FastAPI()

books = [
    {"id": 1, "title": "Clean Code", "category": "software"},
    {"id": 2, "title": "Clean Architecture", "category": "software"},
    {"id": 3, "title": "DDIA", "category": "database"},
]

@app.get("/books")
async def list_books(category: str | None = None, skip: int = 0, limit: int = 10):
    result = books
    if category is not None:
        result = [b for b in books if b["category"].lower() == category.lower()]
    return {"books": result[skip:skip + limit], "total": len(result)}

@app.get("/books/{book_id}")
async def get_book(book_id: int):
    for b in books:
        if b["id"] == book_id:
            return b
    return {"message": "Book not found"}
```

Known limitation (fixed later): Right now, a missing book still returns a normal 200 OK response with a 'not found' message inside it. That's misleading to API clients. We fix this properly with HTTPException in Chapter 7.

10. Common beginner mistakes

- Thinking `/books/25?category=x` has TWO path parameters — no, 25 is the path parameter, `category=x` is a query parameter.
- Forgetting the type hint — `book_id` (no type) doesn't validate anything. `book_id: int` does.
- Turning everything into a query param 'just because' — `GET /books?book_id=25` hides intent; `GET /books/25` communicates it clearly.
- `q: str | None` (no `= None`) still looks optional to a human, but without the default it's actually REQUIRED to FastAPI.
- Returning normal success responses for 'not found' cases instead of a proper 404.

PRACTICE - Answer in your own words

1. What's the difference between a path parameter and a query parameter?

2. Why is `book_id: int` better than just `book_id`?

3. Why must `/users/me` be declared BEFORE `/users/{user_id}`?

4. What do `skip` and `limit` typically control?

5. Why is returning 200 OK with 'not found' text a bad practice?

PRACTICE - Coding exercise — Movie Catalog API

Build a fresh project (movie-catalog-api). Implement:

- GET / -> welcome message
- GET /movies -> list, with `?genre=`, `?skip=`, `?limit=`
- GET /movies/{movie_id} -> single movie by integer ID
- GET /search?q=... -> search by title
- Challenge: add `?short=true` that returns only id/title/genre
- Advanced: GET /movies/{movie_id}/reviews with a `?limit=` query param

Before coding, write out your route table and the shape of each JSON response:

Chapter 3 — Request Bodies & Pydantic Models

1. The problem: our API can READ but not CREATE

Chapter 2 let clients ask for data through the URL. But what if a client wants to SEND a whole new book to be added? Stuffing title, author, price, category, year all into a URL would be ugly and limited. The proper answer: a request body — structured JSON sent along with a POST request.

```
POST /books
{
  "title": "FastAPI in Practice",
  "author": "Jane Developer",
  "year": 2026,
  "price": 49.99
}
```

2. Where does data come from? Three sources

Location	Example	Answers...
Path	/books/25	"WHICH resource?"
Query	?category=software&limit=10	"HOW to filter/behave?"
Body	{ "title": "..." }	"WHAT data to submit?"

FastAPI figures out which is which purely by how you type your function parameters — `int/str/bool` become path or query, and a Pydantic model becomes a body.

3. What is Pydantic, really?

Pydantic is a validation library. Instead of accepting a raw dict and manually checking every field yourself (does 'title' exist? is 'year' really a number?), you write a small class describing the SHAPE the data must have. Pydantic checks incoming data against that shape automatically.

```
from pydantic import BaseModel

class Book(BaseModel):
    title: str
    author: str
    year: int
    price: float
```

- `class Book(BaseModel):` — inheriting from `BaseModel` is what turns a normal class into a Pydantic model.
- Every field WITHOUT a default value (title, author, year, price above) is REQUIRED.
- If the client's JSON is missing 'author', or sends year as text, FastAPI rejects the request before your function even runs.

4. Making a field optional

```
class Book(BaseModel):
    title: str
    author: str
    description: str | None = None # optional, defaults to None
    price: float
```

Careful: `str | None` on its own only describes the TYPE (string or nothing). It's the `= None` default that actually makes the field optional to send. Without the default, it's still required.

5. Your first POST endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

class Book(BaseModel):
    title: str
    author: str
    year: int
    price: float

app = FastAPI()

@app.post("/books")
async def create_book(book: Book):
    return book
```

The key line is `book: Book`. FastAPI sees a Pydantic model as the parameter type, so it knows: read this from the JSON request body, validate it against the `Book` model, and hand me a clean, validated Python object — not a raw dict.

6. `status_code` and 201 Created

```
from fastapi import FastAPI, status

@app.post("/books", status_code=status.HTTP_201_CREATED)
async def create_book(book: Book):
    return book
```

By convention, a successful "create" operation should return 201 Created, not the default 200 OK. `status.HTTP_201_CREATED` is just a readable constant equal to the number 201 — using it instead of a bare `'201'` makes the code self-documenting.

7. `model_dump()` — model to dictionary

```
@app.post("/books")
async def create_book(book: Book):
    book_data = book.model_dump() # -> plain dict
```



```
return {"message": "Book received", "book": book_data}
```

A validated Pydantic object isn't a plain dict by default. `model_dump()` converts it into one — useful whenever you need to pass the data along to another layer (a database call, another function, etc.).

8. Path + Query + Body, all at once

```
class BookUpdate(BaseModel):
    title: str
    price: float

@app.put("/books/{book_id}")
async def update_book(
    book_id: int,          # <- path
    book: BookUpdate,      # <- body
    notify: bool = False   # <- query
):
    return {"book_id": book_id, "notify": notify, "book": book.model_dump()}
```

FastAPI classifies each parameter automatically: matches a {slot} in the path -> path param. Is a Pydantic model -> body. Everything else -> query param.

9. Adding real validation rules with Field

```
from pydantic import BaseModel, Field

class BookCreate(BaseModel):
    title: str = Field(min_length=2, max_length=200)
    year: int = Field(ge=1000, le=2100)
    price: float = Field(gt=0)
    description: str | None = Field(default=None, max_length=500)
```

Constraint	Meaning
min_length / max_length	String length bounds
gt	greater than (strict)
ge	greater than or equal to
lt	less than (strict)
le	less than or equal to

A price of -10 or a title of just 1 character now gets automatically rejected — no manual if-checks needed anywhere in your function body.

10. Response Models — controlling what goes OUT

So far we validated what comes IN. Now: what should the client be allowed to SEE back? Suppose your internal book record has fields like `internal_cost` or `supplier_id` that should never reach the client.

```
class BookResponse(BaseModel):
    id: int
    title: str
    author: str
    price: float

@app.get("/books/{book_id}", response_model=BookResponse)
async def get_book(book_id: int):
    ... # even if the underlying data has extra fields,
        # only id/title/author/price get sent to the client
```

Why this matters: `response_model` isn't just documentation. FastAPI actively filters the returned data down to exactly the fields declared in that model. Any extra internal field is silently dropped before the response leaves your server.

11. Why separate Input and Output models?

A tempting shortcut is one model called `Book` used everywhere. But real systems usually need different shapes at different moments:

Model	Used for	Example fields
<code>BookCreate</code>	Client -> API (creating)	title, author, year, price
<code>BookResponse</code>	API -> Client (returning)	id, title, author, year, price
<code>BookUpdate</code>	Client -> API (updating)	often all-optional versions

Notice `BookCreate` has no `'id'` — the client shouldn't invent its own ID. `BookResponse` DOES have `'id'` — the server generates it and includes it in the response.

12. Full pattern: Book Catalog API v2 (Create + Read)

```
from fastapi import FastAPI, status
from pydantic import BaseModel, Field

class BookCreate(BaseModel):
    title: str = Field(min_length=2, max_length=200)
    author: str = Field(min_length=2, max_length=100)
    year: int = Field(ge=1000, le=2100)
    price: float = Field(gt=0)

class BookResponse(BaseModel):
    id: int
    title: str
    author: str
    year: int
```

```
price: float

app = FastAPI()
books = []

@app.get("/books", response_model=list[BookResponse])
async def list_books():
    return books

@app.post("/books", response_model=BookResponse, status_code=status.HTTP_201_CREATED)
async def create_book(book: BookCreate):
    new_book = {"id": len(books) + 1, **book.model_dump()}
    books.append(new_book)
    return new_book
```

Note on `**book.model_dump()`: The `**` unpacks the dictionary's key-value pairs directly into the new dict literal — it's standard Python, not a FastAPI-specific trick. `{"id": 1, **{"title": "X"}}` becomes `{"id": 1, "title": "X"}`.

13. Common beginner mistakes

- Forgetting `BaseModel` — `'class Book:'` is NOT a Pydantic model, it must be `'class Book(BaseModel):'`.
- Assuming `str | None` alone makes a field optional — you still need `'= None'`.
- Using one model for input, output, AND database storage — fine for a toy project, painful once the app grows.
- Forgetting 201 Created on successful POST endpoints.
- Returning internal/sensitive fields because `response_model` wasn't set.

PRACTICE - Answer in your own words

1. What is a request body, and which HTTP methods commonly use one?

2. What does `Field(gt=0)` mean, in your own words?

3. Why should `BookCreate` and `BookResponse` usually be two different models?

4. What does `response_model` actually DO to the returned data?

5. What does `book.model_dump()` give you back?

PRACTICE - Coding exercise — Movie Catalog API v2

Extend your Chapter 2 movie project (or start fresh) with:

- A `MovieCreate` model: title, director, genre, year, duration, rating, optional description
- A separate `MovieResponse` model that includes an id
- `POST /movies` returning 201 Created
- `Field()` validation: year range, duration > 0, rating in a sensible range
- Challenge: add an `internal_notes` field to stored data but make sure `MovieResponse` never exposes it

Sketch both models and the `POST` endpoint here:

Chapter 4 — Deeper Validation & Splitting the Project into Layers

1. What's new in this chapter

Chapter 3 introduced Pydantic models and `Field()`. Chapter 4 pushes two ideas further: the modern `Annotated[type, Field(...)]` style for declaring fields, and — just as important — NOT cramming everything into one `main.py` file anymore. We start splitting the app into schemas / services / routes.

2. The Annotated style

You'll increasingly see FastAPI code written like this instead of the plain `'field: type = Field(...)'` style:

```
from typing import Annotated
from pydantic import BaseModel, Field

class ProductCreate(BaseModel):
    name: Annotated[str, Field(min_length=3, max_length=100)]
    price: Annotated[float, Field(gt=0)]
    stock: Annotated[int, Field(ge=0)]
```

Read `Annotated[str, Field(...)]` as: "the TYPE is `str`, PLUS here's some extra metadata/validation attached to it." It's functionally similar to `'str = Field(...)'` but keeps the type and the validation rules visually separate, which scales better as fields get more complex (e.g. when combined with `Query()` or `Path()` later).

Why this matters going forward: FastAPI's own docs increasingly favor `Annotated` for `Query`, `Path`, and `Body` parameters too — it's worth getting comfortable with the pattern now.

3. Numeric validation constraints, recap

Constraint	Meaning
<code>gt=0</code>	value must be strictly greater than 0
<code>ge=0</code>	value must be 0 or greater
<code>lt=100</code>	value must be strictly less than 100
<code>le=100</code>	value must be 100 or less

Rule of thumb: price should usually be `gt=0` (a free/zero-priced item is unusual), but stock is fine as `ge=0` (zero items in stock is a completely normal state).

4. Why split the project into layers?

A route function doing EVERYTHING — validating, generating IDs, storing data, formatting the response — works for a demo, but becomes hard to test and maintain as the app grows. The fix is to give each piece of code ONE job:

Layer	Job	File example
Schema	Defines the SHAPE of data (validation)	app/schemas/product.py
Service	Business logic (what actually happens)	app/services/product_service.py
Route	Wires an HTTP path to a function	app/api/routes/products.py

HTTP Request

|

v

Route (only handles the HTTP-level plumbing)

|

v

Schema (validates the incoming data)

|

v

Service (does the actual work / logic)

|

v

Response

5. Example project structure

```
fastapi-product-api/
├── app/
│   ├── main.py
│   ├── api/routes/products.py
│   ├── schemas/product.py
│   └── services/product_service.py
├── pyproject.toml
└── README.md
```

6. The schema (app/schemas/product.py)

```
from typing import Annotated
from pydantic import BaseModel, Field

class ProductCreate(BaseModel):
    name: Annotated[str, Field(min_length=3, max_length=100)]
    description: Annotated[str | None, Field(default=None, max_length=500)]
    price: Annotated[float, Field(gt=0)]
    category: Annotated[str, Field(min_length=2, max_length=50)]
    stock: Annotated[int, Field(ge=0)]
```

7. The service (app/services/product_service.py)

```
from uuid import uuid4
```

```
from app.schemas.product import ProductCreate
```

```
def create_product(product: ProductCreate) -> dict:  
    product_data = product.model_dump()  
    return {"id": str(uuid4()), **product_data}
```

This function knows NOTHING about FastAPI, HTTP, or routes — it's plain Python business logic. That means it's easy to unit-test on its own later, and easy to reuse from other places (a background job, a CLI script, etc.).

8. The route (app/api/routes/products.py)

```
from fastapi import APIRouter, status  
from app.schemas.product import ProductCreate  
from app.services.product_service import create_product  
  
router = APIRouter(prefix="/products", tags=["Products"])  
  
@router.post("/", status_code=status.HTTP_201_CREATED)  
async def create_product_endpoint(product: ProductCreate):  
    return create_product(product)
```

- APIRouter lets you define routes in a SEPARATE file, then plug them into the main app — instead of writing every @app.get(...) directly on the main FastAPI() object.
- prefix="/products" means every route on this router automatically starts with /products — you don't repeat it each time.
- tags=["Products"] just groups these routes together visually in /docs.

9. Wiring the router into main.py

```
from fastapi import FastAPI  
from app.api.routes.products import router as products_router  
  
app = FastAPI(title="Product Catalog API")  
app.include_router(products_router)
```

app.include_router(...) is the glue step. Without it, FastAPI never learns the routes exist, no matter how correctly they're written in products.py.

10. Path + Query + Body with Annotated, all together

```
from typing import Annotated  
from fastapi import APIRouter, Path, Query, status  
  
@router.put("/{product_id}")  
async def update_product_endpoint(  
    product_id: Annotated[int, Path(title="Product ID", ge=1)],  
    product: ProductCreate,  
    notify: Annotated[bool, Query()] = False,
```



```
):  
    return {  
        "product_id": product_id,  
        "notify": notify,  
        "product": product.model_dump(),  
    }
```

Same three sources as before (path / query / body) — just written with the modern Annotated[type, Path(...)/Query(...)] style, which also lets you add validation directly onto path and query params (e.g. `ge=1` means the `product_id` must be 1 or larger).

11. What happens step by step on a request

Client sends JSON

|
v

Pydantic checks each field (name/price/category/stock)

|
+--

invalid --> 422 error response, function never runs

|
+--

valid --> route function runs --> calls service --> returns response

12. Common beginner mistakes

- Treating request bodies as plain dicts (`data: dict`) instead of a typed Pydantic model — you lose validation, docs, and editor support.
- Putting validation logic manually inside the route (`if len(name) < 3: ...`) instead of letting `Field()` express it declaratively.
- Putting business logic directly in the route function instead of the service layer — it works, but doesn't scale.
- Forgetting `app.include_router(...)` — the router file can be perfect and still not show up in the app.

PRACTICE - Answer in your own words

1. What does `Annotated[str, Field(...)]` give you that plain `'str = Field(...)'` doesn't (conceptually)?

2. Why should business logic live in a service function, not directly in a route?

3. What does `APIRouter(prefix="/products")` do?

4. What's the one line that actually connects a router file to the app?

PRACTICE - Coding exercise — Customer Registration API

Build a NEW project (not a copy of Product API) with this layered structure:

- `schemas/customer.py` — `CustomerCreate`: `full_name`, `email`, `age`, `phone`, `city` (add sensible `Field()` validation to each)
- `services/customer_service.py` — `create_customer()` function
- `api/routes/customers.py` — `POST /customers/`, `GET /customers/{customer_id}`, `GET /customers/?city=...`, `PUT /customers/{customer_id}`
- Test that invalid data (`age=-5`, name too short, missing email) gets rejected automatically

Sketch your `CustomerCreate` schema and folder layout here:

Chapter 5 — Response Models: Never Leak Sensitive Data

1. The real-world mistake this chapter fixes

A very common rookie bug: a user registers, you save their username/email/password to your database, and then you return that ENTIRE saved record back to the client — password included. FastAPI gives you an easy, near-automatic way to prevent this: define what comes IN separately from what goes OUT.

2. In-schema vs Out-schema pattern

Schema	Purpose
Input schema (e.g. UserCreate)	What the CLIENT is allowed to send
Output schema (e.g. UserOut)	What the CLIENT is allowed to receive

You then tell the route: "whatever this function returns, filter it through UserOut before sending it to the client." That's the `response_model` parameter doing the real work.

3. The input schema, with proper validation

```
from typing import Annotated
from pydantic import BaseModel, EmailStr, Field

class UserCreate(BaseModel):
    username: Annotated[
        str,
        Field(min_length=3, max_length=30, pattern=r"^[a-zA-Z0-9_]+$"),
    ]
    email: Annotated[EmailStr, Field()]
    password: Annotated[str, Field(min_length=8, max_length=64)]
```

- `pattern=r"^[a-zA-Z0-9_]+$"` — a regex allowing only letters, digits, and underscores in the username (no spaces or symbols).
- `EmailStr` — a special Pydantic type that automatically checks the string actually LOOKS like a real email address, rejecting things like "not-an-email".
- No manual `'if len(username) < 3: raise ...'` code anywhere — Pydantic handles it, and FastAPI turns a failure into a clean 422 response automatically.

4. The output schema — the actual safety net

```
from pydantic import ConfigDict

class UserOut(BaseModel):
    id: int
    username: str
    email: EmailStr
    is_active: bool = True

    model_config = ConfigDict(from_attributes=True)
```

- Notice: NO password field, and NO hashed_password field here at all. If a field isn't declared in UserOut, FastAPI will never include it in the response — no matter what the underlying stored data contains.
- from_attributes=True lets this schema also read data off of an OBJECT (not just a dict) — this becomes useful once a real database ORM is introduced later in the course.

5. Putting it into a real endpoint

```
from fastapi import APIRouter, status

router = APIRouter(prefix="/users", tags=["Users"])

@router.post(
    "/register",
    response_model=UserOut,
    status_code=status.HTTP_201_CREATED,
)
def register_user(user_in: UserCreate) -> UserOut:
    created_user = create_user(user_in) # service function
    return created_user
```

Even if create_user() internally returns a dict that DOES contain hashed_password (which is realistic — that's what a database row often looks like), response_model=UserOut strips it before the response ever leaves the server.

6. A minimal fake service layer (no DB yet)

```
_fake_users_db: list[dict] = []
_next_id = 1

def create_user(user_in: UserCreate) -> dict:
    global _next_id
    hashed_password = f"hashed_{user_in.password}" # placeholder, not real hashing
    new_user = {
        "id": _next_id,
        "username": user_in.username,
        "email": user_in.email,
        "hashed_password": hashed_password,
        "is_active": True,
    }
    _fake_users_db.append(new_user)
    _next_id += 1
    return new_user
```

Note: hashed_password here is just a placeholder string, not real cryptographic hashing. Proper password hashing is a security topic covered later in the course.

7. Testing it in Swagger UI

- Open /docs, find POST /users/register under the 'Users' tag.
- Click 'Try it out', edit the JSON body, click 'Execute'.
- Check the response body — no password field anywhere. Status should read 201.
- Try a short password like "123" — you should see an automatic 422 error explaining exactly what's wrong, without writing any error-handling code yourself.

8. Lighter-weight shortcuts (use sparingly)

```
@router.get("/{user_id}", response_model=UserOut, response_model_exclude={"is_active"})
def get_user(user_id: int): ...

@router.patch("/{user_id}", response_model=UserOut, response_model_exclude_unset=True)
def update_user(user_id: int): ...
```

`response_model_exclude` drops specific fields for just one route.
`response_model_exclude_unset` is handy for PATCH — it only returns fields the client actually sent/changed.

Best practice: For anything security-sensitive (passwords, tokens, admin flags), prefer a dedicated Out schema like `UserOut` over these shortcuts. It's explicit and much harder to misconfigure by accident.

9. Rules of thumb for this pattern

- DO create a separate output schema for anything touching user accounts, auth, or payments.
- DO name schemas clearly: `UserCreate` / `UserUpdate` / `UserOut`, not `UserSchema1` / `UserSchema2`.
- DO keep the service layer free of FastAPI-specific code — no `HTTPException` there, that belongs at the route/handler level (Chapter 7).
- DON'T reuse the exact same schema for both request AND response "to save time" — this is the single most common cause of accidental data leaks in real APIs.
- DON'T process or hash passwords inside the route function — keep that logic in the service layer.

PRACTICE - Answer in your own words

1. Why is it risky to return a database record directly as an API response?

2. What is the practical difference between an input schema and an output schema?

3. What does `response_model` actually do, behind the scenes?

4. When would you reach for `response_model_exclude` vs. just writing a separate Out schema?

PRACTICE - Coding exercise

- Add `full_name` (optional, max 50 chars) to `UserCreate`, and also expose it in `UserOut`.
- Add `is_admin: bool = False` to the fake DB record, but make sure it's NEVER visible in the API response. Verify in Swagger UI.
- Add `GET /users/{user_id}` that looks up a user by ID from the fake DB and returns it via `UserOut`. Try an ID that doesn't exist and note what error you currently get (we fix this properly in Chapter 7).

Sketch your updated `UserCreate` / `UserOut` here:

Chapter 6 — HTTP Status Codes & Custom Responses

1. Why status codes deserve real attention

Picture calling `GET /users/99` for a user that doesn't exist, and the server calmly replies 200 OK with an empty body. Is that an error? Is the user just blank? A client genuinely cannot tell — the status code lied. A well-built API communicates through its status code just as much as through the response body itself.

2. The five status code categories

Range	Category	Meaning
1xx	Informational	Rarely used directly in APIs
2xx	Success	The request worked
3xx	Redirection	Client should look elsewhere
4xx	Client Error	The client sent something wrong
5xx	Server Error	Something broke on the server

3. Codes you'll use constantly

Code	Name	When to use it
200	OK	Successful GET / PUT / PATCH
201	Created	Successful POST that created a new resource
204	No Content	Success, but nothing to return (e.g. after DELETE)
400	Bad Request	Client sent malformed / invalid data
401	Unauthorized	Client isn't authenticated
403	Forbidden	Authenticated, but not allowed to do this
404	Not Found	The resource doesn't exist
422	Unprocessable Entity	Data failed validation (FastAPI's automatic default)
500	Internal Server Error	Something unexpected broke server-side

Rule of thumb: If unsure which code to use, ask: "If I were the client, what would I want to know just from the status code, before even reading the body?"

4. Setting a FIXED status code (declarative)

```
@router.post("/register", status_code=status.HTTP_201_CREATED)
def register_user(user_in: UserCreate):
    return create_user(user_in)
```

5. Setting a DYNAMIC status code (based on runtime logic)

Sometimes the right status code depends on what actually happens inside the function — you can't decide it in advance. Inject the real Response object as a parameter and set it directly:

```
from fastapi import APIRouter, Response, status

@router.get("/", response_model=list[UserOut])
def get_all_users(response: Response):
    users = list_users()
    if not users:
        response.status_code = status.HTTP_204_NO_CONTENT
        return []
    response.status_code = status.HTTP_200_OK
    return users
```

Real detail: A true 204 response should have no body at all. Some teams simply return 200 with an empty list instead — both patterns are common in practice; follow your team's convention.

6. Custom response classes — when it's NOT JSON

```
from fastapi.responses import PlainTextResponse

@router.get("/health", response_class=PlainTextResponse)
def health_check():
    return "OK"
```

Without `response_class=PlainTextResponse`, FastAPI would wrap "OK" as a JSON string (with quotes) instead of returning it as raw plain text. FastAPI also ships `JSONResponse`, `HTMLResponse`, `RedirectResponse`, `FileResponse`, and `StreamingResponse` for other situations.

7. Adding custom headers

```
@router.get("/", response_model=list[UserOut])
def get_all_users(response: Response):
    users = list_users()
    response.headers["X-Total-Count"] = str(len(users))
    return users
```

- Headers are always plain strings, so an integer count must be wrapped in `str()`.
- Prefixing custom headers with X- (e.g. X-Total-Count, X-Request-ID) is a long-standing convention signaling "this is not a standard HTTP header."
- Headers and the response body are separate channels — setting a header doesn't touch the JSON body at all.

8. The manual way to send an error (before Chapter 7)

```
from fastapi.responses import JSONResponse

@router.get("/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    user = get_user_by_id(user_id)
    if user is None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={"detail": f"User with id {user_id} was not found."},
        )
    return user
```

The catch: This works — it's a genuine 404 with a clear message. But every route that needs a "not found" check now repeats this exact if/return block. Imagine writing that in 20+ endpoints. That repetition is exactly what Chapter 7's exception handling fixes.

9. Common beginner mistakes

- Defaulting everything to 200, even for errors — pick the status code that matches what actually happened.
- Using raw numbers (404) instead of `status.HTTP_404_NOT_FOUND` — the constant form is self-documenting and typo-proof.
- Forcing a non-JSON endpoint (health checks, plain text) into the default JSON envelope instead of using `response_class`.
- Copy-pasting manual `JSONResponse` error blocks across many routes — a strong signal centralized error handling is needed.

PRACTICE - Answer in your own words

1. Conceptually, what's the difference between a 4xx and a 5xx status code?

2. Why might an API return 204 instead of 200 with an empty list?

3. How does injecting Response into a function differ from setting status_code= in the decorator?

4. What's the practical downside of hand-writing JsonResponse for every error case across a large app?

PRACTICE - Coding exercise

- Add DELETE /users/{user_id}: removes the user, returns 204 on success, 404 (manual JsonResponse) if missing.
- Add an X-API-Version header to the /health endpoint.
- Make POST /users/register return an appropriate status + message if the username is already taken (pick the best code from the table above).

Sketch your DELETE endpoint logic here:

Chapter 7 — Proper Exception Handling

1. Where Chapter 6 left off

Chapter 6 handled a missing user with a hand-written 'if user is None: return JsonResponse(...)' block. That's a RETURN-based approach — the function decides the HTTP response right there. It mixes two different concerns: business logic ("does this user exist?") and HTTP formatting ("what status code / JSON shape do I send?"). Scaled to 40 endpoints with 2-3 error cases each, that's 100+ nearly-identical blocks scattered across the codebase.

FastAPI's cleaner alternative: RAISE an exception when something goes wrong, and let ONE centralized piece of code decide how to turn it into an HTTP response.

```
# instead of:
if user is None:
    return JsonResponse(status_code=404, content={"detail": "not found"})

# do:
if user is None:
    raise HTTPException(status_code=404, detail="not found")
```

The real power: you can raise this from ANYWHERE — deep inside a service function, a dependency, even a database call — and FastAPI still catches it and builds the correct HTTP response, without that deep code needing to know anything about JsonResponse.

2. Step 1 — FastAPI's built-in HTTPException

```
from fastapi import APIRouter, HTTPException, status

@router.get("/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    user = get_user_by_id(user_id)
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} was not found.",
        )
    return user
```

As soon as 'raise HTTPException(...)' runs, FastAPI immediately stops the function and builds a proper JSON error response — {"detail": "..."} with the status code you specified. No JsonResponse import, no manually built dict.

3. Step 2 — Custom exception classes for YOUR domain

HTTPException is generic — it doesn't know anything about your app's specific concepts. Custom exceptions describe what actually went wrong, in plain business terms, with zero FastAPI/HTTP knowledge baked in:

```
class UserNotFoundError(Exception):
    def __init__(self, user_id: int):
```

```
self.user_id = user_id
super().__init__(f"User with id {user_id} was not found.")
```

```
class UsernameAlreadyExistsError(Exception):
    def __init__(self, username: str):
        self.username = username
        super().__init__(f"Username '{username}' is already taken.")
```

These are PLAIN Python exceptions. Your service layer stays completely free of FastAPI imports — it just describes what went wrong, in its own vocabulary:

```
def get_user_by_id(user_id: int) -> dict:
    for user in _fake_users_db:
        if user["id"] == user_id:
            return user
    raise UserNotFoundError(user_id) # no more returning None!
```

Why this is better: `get_user_by_id` no longer returns `None` for a missing user. A caller can no longer FORGET to check for `None` and accidentally pass a missing user further down the pipeline — the exception forces the issue to surface immediately.

At this point though, if you call the route and the user is missing, you'll actually get a 500 Internal Server Error — because FastAPI doesn't yet know what a `UserNotFoundError` MEANS or which HTTP status it maps to. That's what step 3 fixes.

4. Step 3 — Global exception handlers

```
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse

app = FastAPI()

@app.exception_handler(UserNotFoundError)
def user_not_found_handler(request: Request, exc: UserNotFoundError):
    return JSONResponse(
        status_code=status.HTTP_404_NOT_FOUND,
        content={"error_code": "USER_NOT_FOUND", "detail": str(exc)},
    )

@app.exception_handler(UsernameAlreadyExistsError)
def username_exists_handler(request: Request, exc: UsernameAlreadyExistsError):
    return JSONResponse(
        status_code=status.HTTP_409_CONFLICT,
        content={"error_code": "USERNAME_TAKEN", "detail": str(exc)},
    )
```

- `@app.exception_handler(UserNotFoundError)` registers a function that runs AUTOMATICALLY, anywhere in the whole app, the moment a `UserNotFoundError` is raised — router, service, or three layers deeper.

- A handler always receives (request, exc) — that's a fixed signature FastAPI expects.
- str(exc) reuses the message already built inside the exception's __init__ — no duplication.
- 409 Conflict (not 400) is used for the duplicate username — the request itself is well-formed, it just conflicts with existing data. That's a more precise signal than a generic 400.
- The payoff: if tomorrow every error response needs a new field, you edit it in ONE or TWO handler functions — not a hundred scattered routes.

5. Now the router is refreshingly clean

```
@router.get("/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    return get_user_by_id(user_id) # that's it.
```

If something's wrong, the service layer raises. The route doesn't need any if-checks at all — the exception just bubbles up and the global handler takes care of the rest.

6. Step 4 — customizing the automatic 422 validation error

```
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
def validation_exception_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={"error_code": "VALIDATION_ERROR", "detail": exc.errors()},
    )
```

RequestValidationError is what FastAPI raises internally whenever incoming JSON fails Pydantic validation. Handling it yourself lets you reshape it into your own consistent error_code/detail format, matching the rest of your API's error responses. exc.errors() still gives you Pydantic's detailed per-field breakdown of what went wrong.

7. Step 5 — a catch-all safety net

```
@app.exception_handler(Exception)
def unhandled_exception_handler(request: Request, exc: Exception):
    print(f"Unhandled error: {exc!r}") # log internally (use real logging later)
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={
            "error_code": "INTERNAL_SERVER_ERROR",
            "detail": "Something went wrong on our end. Please try again later.",
        },
    )
```

Important security point: This is your last line of defense against bugs you didn't anticipate. It matches ANY exception not already caught by a more specific handler. Log the real error internally, but NEVER expose raw stack traces, file paths, or internal exception text to the client — that's genuinely useful information for an attacker.

8. Full request flow with exception handling in place

Client Request

|

v

Route --calls--> Service

|

|

|

v

|
+-- raises UserNotFoundError / UsernameAlreadyExistsError

|

v

Normal return

Matching @app.exception_handler(...) builds the response

|

|

+-----> HTTP Response <

9. Common beginner mistakes

- Putting business decisions ("is this a 404 or a 409?") inside the service function — that's an HTTP-layer concern, it belongs in the exception handler.
- Importing HTTPException into the service layer — keep the service layer pure Python, raise plain custom exceptions there instead.
- Skipping the catch-all Exception handler in a real project — an unhandled bug should never leak a stack trace to a client.
- Reusing built-in exceptions like ValueError for domain errors — a custom, named exception class is easier to catch precisely and self-documents intent.

PRACTICE - Answer in your own words

1. What's the core difference between manually returning JSONResponse for an error, and raising an exception?

2. Why should a custom exception like UserNotFoundError avoid importing anything from FastAPI?

3. Why register a handler for Exception itself, and why should it always return a generic message to the client?

4. Why 409 Conflict instead of 400 Bad Request for a duplicate username?

5. What problem does an error_code field solve for a frontend developer?

PRACTICE - Coding exercise

- Add an InvalidUserDataError + matching handler returning 400 Bad Request (e.g. reject usernames that are only numbers).
- Rebuild DELETE /users/{user_id} so the SERVICE raises UserNotFoundError instead of manually returning JSONResponse.
- Add a print() inside UserNotFoundError.__init__ AND inside its handler. Call GET /users/999 and write down the exact order the two print statements fire, and why.

Sketch your InvalidUserDataError + handler here:

Full Module — Quick Reference Cheat Sheet

Concept	Syntax
Path parameter	<code>@app.get("/books/{book_id}") -> book_id: int</code>
Query parameter (optional)	<code>q: str None = None</code>
Query parameter (required)	<code>q: str (no default)</code>
Request body	<code>book: BookCreate (a Pydantic model)</code>
Field validation	<code>Field(min_length=2, gt=0, ge=0, le=100)</code>
Modern style	<code>Annotated[str, Field(min_length=2)]</code>
Create status	<code>status_code=status.HTTP_201_CREATED</code>
Response filtering	<code>response_model=UserOut</code>
Dynamic status	<code>inject 'response: Response', set response.status_code</code>
Custom header	<code>response.headers["X-Total-Count"] = str(n)</code>
Quick error	<code>raise HTTPException(status_code=404, detail="...")</code>
Domain exception	<code>class UserNotFoundError(Exception): ...</code>
Global handler	<code>@app.exception_handler(UserNotFoundError)</code>
Wire a router	<code>app.include_router(users.router)</code>

How the chapters connect: Ch1: app + routes -> Ch2: path/query params -> Ch3: request bodies + Pydantic -> Ch4: deeper validation + layered folders -> Ch5: hiding sensitive fields with `response_model` -> Ch6: correct status codes -> Ch7: raising exceptions instead of manual error returns. Each chapter reuses and upgrades the same running project.